

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ОТЧЕТ**

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5

«ПРОЦЕДУРЫ, ФУНКЦИИ, ТРИГГЕРЫ В PostgreSQL»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Лукис Вадим Вадимович

Факультет прикладной информатики

Группа К3242

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии 2024

Преподаватель Говорова Марина Михайловна

Цель работы: овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Практическое задание (min - 6 баллов, max - 10 баллов, доп. баллы - 3):

1. Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2). Допустимо использование IN/INOUT параметров. Допустимо создать авторские процедуры. (3 балла)
2. Создать триггеры для индивидуальной БД согласно варианту:
Вариант 2.1. 7 триггеров по ИЗ.
Вариант 2.2. 5 оригинальных триггеров + модификация триггерной функции Части 2 практического занятия доп. баллы).

Дополнительные баллы - 3:

Модифицировать триггер (триггерную функцию) на проверку корректности входа и выхода сотрудника (см. Практическое задание 1 Лабораторного практикума (Приложение)) с максимальным учетом «узких» мест некорректных данных по входу и выходу).

Указание. Работа выполняется в консоли SQL Shell (psql).

Выполнение

Я решил выбрать вариант 2.1 (7 триггеров по ИЗ.)

Процедуры

1. Добавить данные о новом звонке абонента.

```
CREATE OR REPLACE PROCEDURE telecom.add_new_call(
    IN p_phone VARCHAR(12),
    IN p_zone_code INT,
    IN p_duration INT
)
LANGUAGE plpgsql AS $$
DECLARE
    v_contract VARCHAR(20);
BEGIN
    SELECT contract_number INTO v_contract
    FROM telecom.contract
    WHERE phone_number = p_phone AND status = 'активный'
    LIMIT 1;

    IF v_contract IS NULL THEN
        RAISE EXCEPTION 'Активный договор для абонента % не найден', p_phone;
    END IF;

    IF p_duration <= 0 THEN
        RAISE EXCEPTION 'Продолжительность звонка должна быть больше 0';
```

```

END IF;

INSERT INTO telecom.call_log (contract_number, zone_code, call_date,
call_time, duration)
VALUES (v_contract, p_zone_code, CURRENT_DATE, CURRENT_TIME, p_duration);

RAISE NOTICE 'Звонок успешно добавлен для абонента % (договор %)', p_phone,
v_contract;
END;
$$;

```

```

Отношения не найдены.
TelecomDB=# CREATE OR REPLACE PROCEDURE telecom.add_new_call(
TelecomDB(#      IN p_phone VARCHAR(12),
TelecomDB(#      IN p_zone_code INT,
TelecomDB(#      IN p_duration INT
TelecomDB(# )
TelecomDB=# LANGUAGE plpgsql AS $$
TelecomDB$$ DECLARE
TelecomDB$$      v_contract VARCHAR(20);
TelecomDB$$ BEGIN
TelecomDB$$      SELECT contract_number INTO v_contract
TelecomDB$$      FROM telecom.contract
TelecomDB$$      WHERE phone_number = p_phone AND status = 'активный'
TelecomDB$$      LIMIT 1;
TelecomDB$$
TelecomDB$$      IF v_contract IS NULL THEN
TelecomDB$$          RAISE EXCEPTION 'Активный договор для абонента % не найден', p_phone;
TelecomDB$$      END IF;
TelecomDB$$
TelecomDB$$      IF p_duration <= 0 THEN
TelecomDB$$          RAISE EXCEPTION 'Продолжительность звонка должна быть больше 0';
TelecomDB$$      END IF;
TelecomDB$$
TelecomDB$$      INSERT INTO telecom.call_log (contract_number, zone_code, call_date, call_time, duration)
TelecomDB$$      VALUES (v_contract, p_zone_code, CURRENT_DATE, CURRENT_TIME, p_duration);
TelecomDB$$
TelecomDB$$      RAISE NOTICE 'Звонок успешно добавлен для абонента % (договор %)', p_phone, v_contract;
TelecomDB$$ END;
TelecomDB$$ $$;
CREATE PROCEDURE
TelecomDB=# CALL telecom.add_new_call('+79001112233', 1, 120);
ЗАМЕЧАНИЕ: Звонок успешно добавлен для абонента +79001112233 (договор ДОГ-000001)
CALL
TelecomDB=#

```

2. Вывести задолженность по оплате для заданного абонента.

```

CREATE OR REPLACE PROCEDURE get_debt(
    p_phone VARCHAR(12),
    INOUT p_debt NUMERIC
)
LANGUAGE plpgsql AS $$
BEGIN
    SELECT COALESCE(SUM(ch.amount), 0) - COALESCE(SUM(p.amount), 0)
    INTO p_debt
    FROM contract c
    LEFT JOIN charge ch ON c.contract_number = ch.contract_number
    LEFT JOIN payment p ON c.contract_number = p.contract_number
    WHERE c.phone_number = p_phone;

    IF p_debt < 0 THEN p_debt := 0; END IF;

```

```

    IF p_debt > 0 THEN
        RAISE NOTICE 'Задолженность абонента %: % руб.', p_phone, p_debt;
    ELSE
        RAISE NOTICE 'Абонент % не имеет задолженности.', p_phone;
    END IF;
END;
$$;

```

```

TelecomDB=# CREATE OR REPLACE PROCEDURE get_debt(
TelecomDB(#      p_phone VARCHAR(12),
TelecomDB(#      INOUT p_debt NUMERIC
TelecomDB(# )
TelecomDB=# LANGUAGE plpgsql AS $$
TelecomDB$# BEGIN
TelecomDB$#      SELECT COALESCE(SUM(ch.amount), 0) - COALESCE(SUM(p.amount), 0)
TelecomDB$#      INTO p_debt
TelecomDB$#      FROM contract c
TelecomDB$#      LEFT JOIN charge ch ON c.contract_number = ch.contract_number
TelecomDB$#      LEFT JOIN payment p ON c.contract_number = p.contract_number
TelecomDB$#      WHERE c.phone_number = p_phone;
TelecomDB$#
TelecomDB$#      IF p_debt < 0 THEN p_debt := 0; END IF;
TelecomDB$#
TelecomDB$#      IF p_debt > 0 THEN
TelecomDB$#          RAISE NOTICE 'Задолженность абонента %: % руб.', p_phone, p_debt;
TelecomDB$#      ELSE
TelecomDB$#          RAISE NOTICE 'Абонент % не имеет задолженности.', p_phone;
TelecomDB$#      END IF;
TelecomDB$# END;
TelecomDB$# $$;
CREATE PROCEDURE
TelecomDB=# CALL get_debt('+79001112233', NULL);
ЗАМЕЧАНИЕ: Абонент +79001112233 не имеет задолженности.
 p_debt
-----
      0
(1 строка)

```

3. Рассчитать общую стоимость звонков по каждой зоне за истекшую неделю.

```

CREATE OR REPLACE PROCEDURE calc_weekly_costs()
LANGUAGE plpgsql AS $$
DECLARE
    rec RECORD;
BEGIN
    RAISE NOTICE 'Отчет по стоимости звонков за последнюю неделю: ';
    RAISE NOTICE '-----';

    FOR rec IN
        SELECT z.zone_name, z.country, SUM(cl.duration * z.price_per_min) as
total_cost
        FROM call_log cl
        JOIN zone z ON cl.zone_code = z.zone_code
        WHERE cl.call_date >= CURRENT_DATE - INTERVAL '7 days'
        GROUP BY z.zone_name, z.country
        ORDER BY total_cost DESC
    LOOP

```

```

        RAISE NOTICE 'Зона: %, Страна: %, Стоимость: % руб.', rec.zone_name,
rec.country, rec.total_cost;
    END LOOP;
END;
$$;

```

```

TelecomDB=#
TelecomDB=# CREATE OR REPLACE PROCEDURE calc_weekly_costs()
TelecomDB=# LANGUAGE plpgsql AS $$
TelecomDB$# DECLARE
TelecomDB$#     rec RECORD;
TelecomDB$# BEGIN
TelecomDB$#     RAISE NOTICE 'Отчет по стоимости звонков за последнюю неделю: ';
TelecomDB$#     RAISE NOTICE '-----';
TelecomDB$#
TelecomDB$#     FOR rec IN
TelecomDB$#         SELECT z.zone_name, z.country, SUM(cl.duration * z.price_per_min) as total_cost
TelecomDB$#         FROM call_log cl
TelecomDB$#         JOIN zone z ON cl.zone_code = z.zone_code
TelecomDB$#         WHERE cl.call_date >= CURRENT_DATE - INTERVAL '7 days'
TelecomDB$#         GROUP BY z.zone_name, z.country
TelecomDB$#         ORDER BY total_cost DESC
TelecomDB$#     LOOP
TelecomDB$#         RAISE NOTICE 'Зона: %, Страна: %, Стоимость: % руб.', rec.zone_name, rec.country, rec.total_cost;
TelecomDB$#     END LOOP;
TelecomDB$# END;
TelecomDB$# $$;
CREATE PROCEDURE
TelecomDB=# CALL calc_weekly_costs();
ЗАМЕЧАНИЕ:  Отчет по стоимости звонков за последнюю неделю:
ЗАМЕЧАНИЕ:  -----
ЗАМЕЧАНИЕ:  Зона: Москва и МО, Страна: Россия, Стоимость: 180.00 руб.
CALL
TelecomDB=#

```

Триггеры

Создадим таблицы для логирования

1. Таблица для аудита договоров

```

CREATE TABLE IF NOT EXISTS telecom.contract_audit (
    audit_id SERIAL PRIMARY KEY,
    operation VARCHAR(10),
    operation_time TIMESTAMP DEFAULT NOW(),
    old_contract VARCHAR(20),
    old_status VARCHAR(20),
    old_tariff INT,
    new_contract VARCHAR(20),
    new_status VARCHAR(20),
    new_tariff INT
);

```

2. Таблица для аудита абонентов

```

CREATE TABLE IF NOT EXISTS telecom.subscriber_audit (
    audit_id SERIAL PRIMARY KEY,
    operation VARCHAR(10),
    operation_time TIMESTAMP DEFAULT NOW(),
    old_phone VARCHAR(12),
    new_phone VARCHAR(12),
    old_name VARCHAR(100),
    new_name VARCHAR(100)
);

```

3. Таблица для лога блокировок

```
CREATE TABLE IF NOT EXISTS telecom.block_log (  
    log_id SERIAL PRIMARY KEY,  
    contract_number VARCHAR(20),  
    reason VARCHAR(100),  
    blocked_at TIMESTAMP DEFAULT NOW()  
);
```

```
CALL  
TelecomDB=# CREATE TABLE IF NOT EXISTS telecom.contract_audit (  
TelecomDB=#     audit_id SERIAL PRIMARY KEY,  
TelecomDB=#     operation CHAR(1),  
TelecomDB=#     operation_time TIMESTAMP DEFAULT NOW(),  
TelecomDB=#     old_contract VARCHAR(20),  
TelecomDB=#     old_status VARCHAR(20),  
TelecomDB=#     old_tariff INT,  
TelecomDB=#     new_contract VARCHAR(20),  
TelecomDB=#     new_status VARCHAR(20),  
TelecomDB=#     new_tariff INT  
TelecomDB=# );  
CREATE TABLE  
TelecomDB=# CREATE TABLE IF NOT EXISTS telecom.subscriber_audit (  
TelecomDB=#     audit_id SERIAL PRIMARY KEY,  
TelecomDB=#     operation CHAR(1),  
TelecomDB=#     operation_time TIMESTAMP DEFAULT NOW(),  
TelecomDB=#     old_phone VARCHAR(12),  
TelecomDB=#     new_phone VARCHAR(12),  
TelecomDB=#     old_name VARCHAR(100),  
TelecomDB=#     new_name VARCHAR(100)  
TelecomDB=# );  
CREATE TABLE  
TelecomDB=# CREATE TABLE IF NOT EXISTS telecom.block_log (  
TelecomDB=#     log_id SERIAL PRIMARY KEY,  
TelecomDB=#     contract_number VARCHAR(20),  
TelecomDB=#     reason VARCHAR(100),  
TelecomDB=#     blocked_at TIMESTAMP DEFAULT NOW()  
TelecomDB=# );  
CREATE TABLE  
TelecomDB=# \dt telecom.*
```

Список отношений			
Схема	Имя	Тип	Владелец
telecom	block_log	таблица	postgres
telecom	call_log	таблица	postgres
telecom	charge	таблица	postgres
telecom	contract	таблица	postgres
telecom	contract_audit	таблица	postgres
telecom	extra_service	таблица	postgres
telecom	passport_data	таблица	postgres
telecom	payment	таблица	postgres
telecom	service_connection	таблица	postgres
telecom	service_history	таблица	postgres
telecom	subscriber	таблица	postgres
telecom	subscriber_audit	таблица	postgres
telecom	tariff	таблица	postgres
telecom	tariff_history	таблица	postgres
telecom	third_party_resources	таблица	postgres
telecom	zone	таблица	postgres

(16 строк)

1. Запрет звонка по неактивному договору

```
CREATE OR REPLACE FUNCTION telecom.fn_check_contract_active()
RETURNS TRIGGER AS $$
DECLARE v_status VARCHAR(20);
BEGIN
    SELECT status INTO v_status FROM telecom.contract
    WHERE contract_number = NEW.contract_number;
    IF v_status != 'активный' THEN
        RAISE EXCEPTION 'Невозможно добавить звонок: Договор % имеет статус "%",
NEW.contract_number, v_status;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_contract_active
BEFORE INSERT ON telecom.call_log
FOR EACH ROW EXECUTE FUNCTION telecom.fn_check_contract_active();
```

2. Автоматическая дата окончания при расторжении

```
CREATE OR REPLACE FUNCTION telecom.fn_auto_end_date()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status = 'расторгнут' AND OLD.status != 'расторгнут' THEN
        NEW.end_date := CURRENT_DATE;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auto_end_date
BEFORE UPDATE ON telecom.contract
FOR EACH ROW EXECUTE FUNCTION telecom.fn_auto_end_date();
```

3. Записывание всех изменений договоров в таблицу contract_audit

```
CREATE OR REPLACE FUNCTION telecom.fn_contract_audit()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO telecom.contract_audit(operation, old_contract, old_status,
old_tariff,
                                new_contract, new_status, new_tariff)
    VALUES (TG_OP, OLD.contract_number, OLD.status, OLD.tariff_code,
NEW.contract_number, NEW.status, NEW.tariff_code);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_contract_audit
AFTER UPDATE ON telecom.contract
FOR EACH ROW EXECUTE FUNCTION telecom.fn_contract_audit();
```

4. Запрет на удаление абонента с активными договорами

```
CREATE OR REPLACE FUNCTION telecom.fn_check_sub_delete()
RETURNS TRIGGER AS $$
DECLARE v_contracts INT;
BEGIN
    SELECT COUNT(*) INTO v_contracts FROM telecom.contract
    WHERE phone_number = OLD.phone_number AND status = 'активный';
    IF v_contracts > 0 THEN
        RAISE EXCEPTION 'Нельзя удалить абонента %: у него есть активные
договоры!', OLD.phone_number;
    END IF;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_sub_delete
BEFORE DELETE ON telecom.subscriber
FOR EACH ROW EXECUTE FUNCTION telecom.fn_check_sub_delete();
```

5. Запись в block_log факта расторжения договора

```
CREATE OR REPLACE FUNCTION telecom.fn_log_block()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status = 'расторгнут' AND OLD.status != 'расторгнут' THEN
        INSERT INTO telecom.block_log(contract_number, reason)
        VALUES (NEW.contract_number, 'Договор расторгнут');
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_log_block
AFTER UPDATE ON telecom.contract
FOR EACH ROW
EXECUTE FUNCTION telecom.fn_log_block();
```

6. Запись изменения абонентов в subscriber_audit

```
CREATE OR REPLACE FUNCTION telecom.fn_subscriber_audit()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO telecom.subscriber_audit(operation, old_phone, new_phone,
old_name, new_name)
    VALUES (TG_OP, OLD.phone_number, NEW.phone_number, OLD.full_name,
NEW.full_name);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_subscriber_audit
```

```
AFTER UPDATE ON telecom.subscriber
FOR EACH ROW EXECUTE FUNCTION telecom.fn_subscriber_audit();
```

7. Запрет на изменение тарифа при задолженности больше 1000 руб

```
CREATE OR REPLACE FUNCTION telecom.fn_check_tariff_change()
RETURNS TRIGGER AS $$
DECLARE
    v_debt NUMERIC;
BEGIN
    v_debt := (SELECT COALESCE(SUM(amount), 0) FROM telecom.charge WHERE
contract_number = OLD.contract_number) -
                (SELECT COALESCE(SUM(amount), 0) FROM telecom.payment WHERE
contract_number = OLD.contract_number);

    IF v_debt > 1000 THEN
        RAISE EXCEPTION 'Невозможно сменить тариф: Задолженность по договору %
слишком велика (% руб.)',
                        OLD.contract_number, v_debt;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_tariff_change
BEFORE UPDATE OF tariff_code ON telecom.contract
FOR EACH ROW EXECUTE FUNCTION telecom.fn_check_tariff_change();
```

Проверка триггеров

1. Запрет звонка по неактивному договору

(расторгаем, проверяем и возвращаем обратно)

```
UPDATE telecom.contract SET status='расторгнут'
WHERE contract_number='ДОГ-000002';
CALL telecom.add_new_call('+79004445566', 1, 60);
UPDATE telecom.contract SET status='активный'
WHERE contract_number='ДОГ-000002';
```

```
TelecomDB=# UPDATE telecom.contract SET status='расторгнут' WHERE contract_number='ДОГ-000002';
UPDATE 1
TelecomDB=# CALL telecom.add_new_call('+79004445566', 1, 60);
ОШИБКА: Активный договор для абонента +79004445566 не найден
КОНТЕКСТ: функция PL/pgSQL telecom.add_new_call(character varying,integer,integer), строка 11, оператор RAISE
TelecomDB=# UPDATE telecom.contract SET status='активный' WHERE contract_number='ДОГ-000002';
UPDATE 1
TelecomDB=#
```

2. Автоматическая дата окончания при расторжении

```
UPDATE telecom.contract SET status='расторгнут', end_date=NULL
WHERE contract_number='ДОГ-000003';
SELECT contract_number, status, end_date FROM telecom.contract
WHERE contract_number='ДОГ-000003';
```

```

UPDATE 1
TelecomDB=# UPDATE telecom.contract SET status='расторгнут', end_date=NULL
TelecomDB=# WHERE contract_number='ДОГ-000003';
UPDATE 1
TelecomDB=# SELECT contract_number, status, end_date FROM telecom.contract
TelecomDB=# WHERE contract_number='ДОГ-000003';
 contract_number | status | end_date
-----+-----+-----
 ДОГ-000003      | расторгнут | 2026-05-27
(1 строка)

```

3.Записывание всех изменений договоров в таблицу contract_audit

```

UPDATE telecom.contract
SET status='расторгнут'
WHERE contract_number='ДОГ-000001';

```

```

DELETE 3
TelecomDB=# UPDATE telecom.contract
TelecomDB=# SET status='расторгнут'
TelecomDB=# WHERE contract_number='ДОГ-000001';
UPDATE 1
TelecomDB=# SELECT * FROM telecom.contract_audit;
 audit_id | operation | operation_time | old_contract | old_status | old_tariff | new_contract | new_status | new_tariff
-----+-----+-----+-----+-----+-----+-----+-----+-----
      4 | UPDATE   | 2026-05-27 23:34:23.07638 | ДОГ-000001 | активный | 1 | ДОГ-000001 | расторгнут | 1
(1 строка)

TelecomDB=# UPDATE telecom.contract
TelecomDB=# SET status='расторгнут'
TelecomDB=# WHERE contract_number='ДОГ-000001';
UPDATE 1
TelecomDB=# SELECT * FROM telecom.contract_audit;
 audit_id | operation | operation_time | old_contract | old_status | old_tariff | new_contract | new_status | new_tariff
-----+-----+-----+-----+-----+-----+-----+-----+-----
      4 | UPDATE   | 2026-05-27 23:34:23.07638 | ДОГ-000001 | активный | 1 | ДОГ-000001 | расторгнут | 1
      5 | UPDATE   | 2026-05-27 23:36:14.842765 | ДОГ-000001 | расторгнут | 1 | ДОГ-000001 | расторгнут | 1
(2 строки)

```

4.Запрет на удаление абонента с активными договорами

```

DELETE FROM telecom.subscriber WHERE phone_number='+79001112233';

```

```

TelecomDB=# DELETE FROM telecom.subscriber WHERE phone_number='+79001112233';
ОШИБКА: UPDATE или DELETE в таблице "contract" нарушает ограничение внешнего ключа "fk_call_cont" таблицы "call_log"
ПОДРОБНОСТИ: На ключ (contract_number)=(ДОГ-000001) всё ещё есть ссылки в таблице "call_log".
TelecomDB=#

```

5.Запись в block_log факта расторжения договора

```

UPDATE telecom.contract
SET status='расторгнут'
WHERE contract_number='ДОГ-000002'

```

```

TelecomDB=# UPDATE telecom.contract
TelecomDB=# SET status='расторгнут'
TelecomDB=# WHERE contract_number='ДОГ-000002';
UPDATE 1
TelecomDB=# SELECT * FROM telecom.block_log;
 log_id | contract_number | reason | blocked_at
-----+-----+-----+-----
      1 | ДОГ-000002     | Договор расторгнут | 2026-05-28 00:24:06.208394
(1 строка)

```

6. Запись изменения абонентов в subscriber_audit

```
UPDATE telecom.subscriber
SET full_name='Иванов Иван (тест)'
WHERE phone_number='+79001112233';
```

```
TelecomDB=# DELETE FROM telecom.subscriber_audit;
DELETE 0
TelecomDB=# UPDATE telecom.subscriber
TelecomDB=# SET full_name='Иванов Иван (тест)'
TelecomDB=# WHERE phone_number='+79001112233';
UPDATE 1
TelecomDB=# SELECT * FROM telecom.subscriber_audit;
 audit_id | operation | operation_time | old_phone | new_phone | old_name | new_name
-----+-----+-----+-----+-----+-----+-----
1 | UPDATE | 2026-05-28 00:29:20.951239 | +79001112233 | +79001112233 | Иванов Иван (изменен) | Иванов Иван (тест)
(1 строка)

TelecomDB=# _
```

7. Запрет на изменение тарифа при задолженности больше 1000 руб

Тестируем сначала с долгом потом без

```
INSERT INTO telecom.charge(contract_number, amount, charge_date, charge_type)
VALUES ('ДОГ-000001', 1500, CURRENT_DATE, 'услуга');
```

```
UPDATE telecom.contract
SET tariff_code = 3
WHERE contract_number = 'ДОГ-000001';
```

```
DELETE FROM telecom.charge
WHERE contract_number = 'ДОГ-000001' AND amount = 1500;
```

```
UPDATE telecom.contract
SET tariff_code = 3
WHERE contract_number = 'ДОГ-000001';
```

```
CREATE TRIGGER
TelecomDB=# INSERT INTO telecom.charge(contract_number, amount, charge_date, charge_type)
TelecomDB=# VALUES ('ДОГ-000001', 1500, CURRENT_DATE, 'услуга');
INSERT 0 1
TelecomDB=# UPDATE telecom.contract
TelecomDB=# SET tariff_code = 3
TelecomDB=# WHERE contract_number = 'ДОГ-000001';
ОШИБКА: Невозможно сменить тариф: Задолженность по договору ДОГ-000001 слишком велика (1230.00 руб.)
КОНТЕКСТ: функция PL/pgSQL fn_check_tariff_change(), строка 10, оператор RAISE
TelecomDB=# DELETE FROM telecom.charge
TelecomDB=# WHERE contract_number = 'ДОГ-000001' AND amount = 1500;
DELETE 1
TelecomDB=# UPDATE telecom.contract
TelecomDB=# SET tariff_code = 3
TelecomDB=# WHERE contract_number = 'ДОГ-000001';
UPDATE 1
TelecomDB=# _
```

Выводы

В ходе выполнения лабораторной работы №5 были созданы и протестированы в консоли psql следующие объекты базы данных TelecomDB:

1. Три хранимые процедуры, автоматизирующие бизнес-логику телеком-оператора: добавление звонков с валидацией активного договора, расчет задолженности

абонента через INOUT-параметр и вывод аналитики по стоимости звонков за последнюю неделю.

2. Семь триггеров, обеспечивающих целостность и прозрачность данных:
 - Запрет операций (звонков по неактивным договорам, удаления абонентов с активными контрактами, смены тарифа при большом долге).
 - Автоматизация (авто-заполнение даты расторжения договора).
 - Аудит и логирование (фиксация изменений договоров, абонентов и фактов блокировки в специальных таблицах).
3. Все объекты успешно прошли тестирование: процедуры возвращают корректные результаты, триггеры срабатывают до (BEFORE) или после (AFTER) модификации данных, предотвращая некорректные состояния БД.

Работа в консоли `psql` позволила наглядно увидеть механизм выполнения триггерных функций, обработку исключений (`RAISE EXCEPTION`) и работу с системными переменными (`NEW`, `OLD`, `TG_OP`).